

System Overview: architecture-model-standard

System Purpose

architecture-model-standard is a software system that provides a universal, machine-readable Architecture-as-Code standard. It defines a YAML schema for describing software system architectures and provides tooling to parse, validate, slice, and generate architectural models. The system operates on schema version 1.5.

Architecture Overview

The system is organized as a single **Application Layer** (L-APP) containing all components. This flat architecture reflects the system's nature as a focused library and CLI tool rather than a distributed application. All nine top-level components reside at the same architectural tier, with the CLI serving as the entry point and the core, manifest, config, spec, extract, profiles, utils, and enrich components providing the underlying functionality.

Key Components

Component	Role	Type
core	Central processing engine providing parsing, validation, and slicing of architecture models	Service
manifest	Reality Manifest generator that produces ground-truth inventories via AST scanning	Service
config	Configuration loading and auto-discovery for project analysis	Service
spec	JSON Schema definitions used for model validation	Service
cli	Command-line interface exposing all system capabilities to users	Service
extract	Extracts architecture models from generated Tier 1 artifacts	Service
profiles	Domain profile system for cross-domain architecture modeling	Service
utils	Shared utilities for file discovery and exclusion patterns	Service
enrich	Enriches architecture models with AST-level detail (body hints, constants, test contracts)	Service

Key Relationships

Dependency Structure

The **cli** component acts as the top-level orchestrator, depending on three other components: - **core** — for parsing, validation, and slicing operations - **config** — for configuration loading - **manifest** — for manifest generation

The **core** component depends on: - **config** — for configuration access - **spec** — for schema definitions used during validation

The **manifest** component depends on: - **config** — for project configuration and discovery

Exposed Interfaces

Interface	Exposed By	Consumed By
CLI Interface	cli	Developer
Parser API	core	LLM Agent
Validator API	core	—
Slicer API	core	LLM Agent
Manifest API	manifest	—

Actor Interactions

- **Developers** interact with the system through the CLI Interface, which provides 9 commands for initialization, validation, slicing, diffing, statistics, impact analysis, manifest generation, coverage, and enrichment.
- **LLM Agents** consume the Parser API and Slicer API directly, enabling programmatic loading and querying of architecture models for AI-driven system engineering workflows.

Constraints

- The **core** and **spec** components are constrained by the schema definition (CON-SCHEMA), ensuring all operations conform to the defined architecture model format.
- The **core** component is additionally constrained by a no-orphans rule (CON-NO-ORPHANS), ensuring all entities in a model are connected through relationships.